SENTINEL: A Local-LLM Detection-to-Response Pipeline for Home IoT, and the Evaluation Pitfalls It Exposed

Vinay [SURNAME — confirm]
[Department — confirm]
[University — confirm]
[City, Country — confirm]
[e-mail — confirm]

***Abstract*—Home IoT devices run unattended, with no one to defend them. We present SENTINEL, an open-source appliance that detects anomalous traffic with a sequence autoencoder over Kitsune features, labels events with MITRE ATT&CK techniques using a language model running on the appliance, and enforces blocks through nftables under a deterministic, auditable policy, with honeypots feeding the same path. We then evaluate the whole system rigorously. On N-BaIoT, with entire devices held out of training, the detector catches eight of ten attack classes almost perfectly yet misses nearly all BASHLITE TCP and UDP floods, 27.8% of attack windows, for which its scores barely separate attack from benign traffic; false positives rise from $0.23\% \pm 0.02\%$ to $3.65\% \pm 0.12\%$ on unseen devices, driven mostly by one device. The blind spot is shared by a simpler autoencoder, which matches the sequence model at lower cost. In a pre-registered test the local LLM named a correct ATT&CK technique for 0.0% [0.0, 1.9] of attack windows, labelling almost everything as command-and-control even when given a traffic summary, while its high severities would still have triggered blocks for 81.0% [75.0, 85.8] of attacks. We also found that SENTINEL's live feature extractor computed 40 of 115 features differently from the training data, and fixed it with a parity test built from invariants of the training set. Every in-house stage takes under a millisecond; the path is dominated by the LLM (9.5 s) and a blocking threat-intelligence lookup (median 21.5 s). We release the code, the pre-registration and all results.**

***Index Terms*—IoT security, anomaly detection, large language models, MITRE ATT&CK, evaluation methodology, N-BaIoT**

I. INTRODUCTION

Consumer Internet-of-Things devices are numerous, rarely patched, and frequently exposed on home networks with default credentials. The Mirai botnet showed the consequence at scale: a large population of cameras, routers and recorders was conscripted into denial-of-service attacks largely by guessing factory passwords [1]. A household has no security operations team to watch its traffic. Whatever defends it has to run unattended, on modest hardware, and without shipping private traffic to a third party.

This paper describes SENTINEL, an open-source appliance that attempts exactly that, and reports what a careful evaluation of it revealed. SENTINEL captures traffic on the home segment, scores it with an autoencoder over Kitsune's streaming features [2], asks a locally hosted large language model (LLM) to label anomalous events with MITRE ATT&CK tactics and techniques [3], and turns the label into an nftables [4] block through a deterministic, auditable response policy. Honeypots [5], [6] feed attacker addresses into the same enforcement path. The language model runs on the appliance itself, so no packet payload or traffic content leaves the home.

Building such a pipeline is now easy; knowing whether it works is not. Anomaly detectors for IoT traffic are routinely reported at near-perfect accuracy on public benchmarks, yet aggregate scores on lab datasets say little about how a system behaves when deployed [7], [8]. We therefore treated evaluation as the main object of study, and it overturned several things we believed about our own system:

- *A per-class blind spot hidden by the aggregate.* On N-BaIoT [9], the deployed detector catches the Mirai attacks almost perfectly, yet misses nearly all BASHLITE TCP and UDP floods, which make up 27.8% of attack windows. An aggregate detection rate conceals this entirely.
- *False positives grow on unseen devices.* Holding out entire devices from training, as a new gadget joining a home network would be, raises the false-positive rate from $0.23\% \pm 0.02\%$ to $3.65\% \pm 0.12\%$ for our reproducible model.
- *The live feature extractor did not match the training features.* SENTINEL's own re-implementation of the Kitsune extractor computed 40 of the 115 features differently from the data the model was trained on, so offline accuracy did not describe the running system. We traced the discrepancy, fixed it by adopting the reference implementation, and verified parity against invariants measured on the training data itself.
- *The LLM's labels were almost never right.* In a pre-registered test, the local model named a correct ATT&CK technique for 0.0% [0.0, 1.9] of attack windows, labelling 90.8% of all events as Command and Control; describing the traffic to it did not help. Because the response policy acts on the model's severity, which it set high for most events, the pipeline would still have blocked 81.0% [75.0, 85.8] of attacks—for reasons unrelated to the label.

Our contributions are:

1) The design and open-source implementation of an end-to-end, on-premise detection-to-response pipeline for home IoT networks that keeps inference, including the LLM, on the appliance (§III).
2) A leakage-aware evaluation protocol for N-BaIoT with a *held-out-device* split, applied to the deployed model, a reproducible retraining over five seeds, and two baselines (§IV, §V-A).
3) A pre-registered evaluation of LLM ATT&CK labelling on the events the pipeline actually produces, with and without a traffic summary, showing that a small local model does not yet perform this task (§V-D).
4) A per-stage latency breakdown on a four-core aarch64 host, locating the bottleneck of the detection-to-response path (§V-E).
5) Practical lessons for building and evaluating such systems: recovering the provenance of an undocumented model from its scaler, and checking that live features match training features (§VI).

All code, the evaluation harness, the pre-registration and the result files are released with the system.[1]

II. Background and Related Work

a) Anomaly detection for IoT traffic: Kitsune introduced a family of damped, incremental statistics computed per packet over several time scales and several stream aggregations (host, MAC–IP pair, channel, socket), together with an ensemble of small autoencoders that learns benign behaviour online [2]. It was designed for resource-constrained network devices and evaluated on a Raspberry Pi. N-BaIoT applied the same 115 features to traffic captured from nine commercial IoT devices before and after infection by the Mirai and BASHLITE botnets, training one deep autoencoder per device and reporting that all attacks were detected [9]. Newer benchmarks such as CICIoT2023 cover more devices and attack types [10]. SENTINEL reuses the Kitsune feature definition and the N-BaIoT data, but trains a single sequence autoencoder shared across devices, which is the realistic choice for an appliance that cannot collect labelled benign traffic for every new gadget in advance.

b) How such detectors are evaluated: Sommer and Paxson argued that anomaly detection for networks is harder than the machine-learning framing suggests: the cost of false positives is high, the semantic gap between an anomaly and an attack is wide, and evaluation data rarely resembles deployment [7]. Arp et al. catalogue recurring pitfalls in security machine learning, among them data snooping, spurious correlations, inappropriate baselines and misleading metrics under class imbalance [8], and dataset audits have found labelling and construction errors in widely used intrusion datasets [11]. Our evaluation is an application of that advice to a complete system rather than to a single model; several of the problems we report are instances of the pitfalls they describe.

[1]Repository URL withheld for review.

c) LLMs for security operations: Large language models are increasingly applied to security tasks, including threat-intelligence extraction and mapping to ATT&CK; a recent systematic review surveys this literature [12], and surveys of ATT&CK's use in research describe the mapping problem in detail [13], [14]. Agentic designs that chain detection, classification and response have also appeared; AgenticCyber, for instance, orchestrates cloud-hosted multimodal models over enterprise telemetry [15]. SENTINEL differs in placing a small quantised model on the appliance itself, which removes the need to send traffic to a cloud service but constrains model size and speed. Most prior evaluations of LLM ATT&CK mapping use written threat reports as input; we evaluate the model on the structured anomaly events a detector actually produces.

d) Deception and automated blocking: Honeypots are a long-standing source of attacker intelligence for IoT. HoneyIoT adapts a high-interaction IoT honeypot's responses with reinforcement learning to prolong attacker engagement [16]. Heiding et al. used honeypot observations to evaluate login blocking for IoT devices [17], so feeding deception data into blocking is not new. SENTINEL's contribution here is integration rather than invention: honeypot sessions and detector alerts share one enforcement path with one safety guard.

e) Kernel-level enforcement: eBPF and XDP allow packets to be filtered in the kernel's fast path. iKern embeds intrusion detection and prevention in eBPF [18], Zhang et al. run a neural network inside the kernel for real-time prevention [19], and eBPF-based DDoS mitigation has been demonstrated on a Raspberry Pi 4 [20]. SENTINEL keeps all learning in user space and uses nftables for enforcement; it contains an XDP drop path, but that path was not enabled in the configuration evaluated here.

f) Home IoT context: IoT Inspector collected labelled traffic from smart-home devices at scale and found device behaviour to be comparatively regular [21], which is the premise of anomaly detection on the home segment. Manufacturer Usage Descriptions let vendors declare the traffic a device needs [22]; they are complementary to anomaly detection, but require manufacturer participation that most consumer devices lack.

III. System Design

SENTINEL is a set of cooperating services on one host, connected by a Redis publish/subscribe bus (Fig. 1). We describe the system as built and evaluated, which differs in places from its original design; where it does, we say so.

A. Capture and device context

A capture service reads packets from the monitored interface with Scapy's asynchronous sniffer and publishes a compact per-packet record (addresses, ports, length, timestamp) on the bus. A discovery service builds a device registry passively from the mDNS, SSDP, DHCP and ARP traffic it observes, maps MAC prefixes to vendors, and queries NIST's National Vulnerability Database for known CVEs by vendor. Active

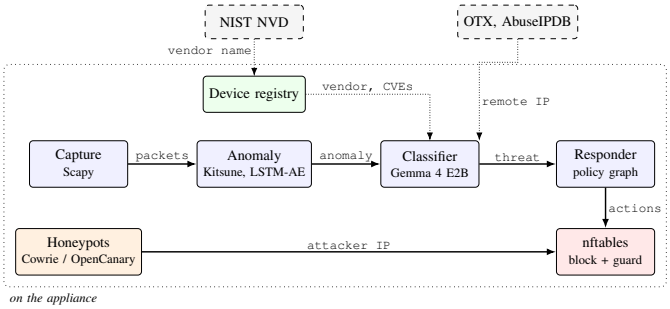


Fig. 1. The detection-to-response path. Solid arrows are bus channels; dashed boxes are the only components that contact external services.

Nmap scanning exists but is triggered only on demand from the dashboard.

B. Anomaly detection

For every packet the anomaly service computes the 115 Kitsune features [2]: damped statistics at five time scales ($\lambda \in \{5, 3, 1, 0.1, 0.01\}$) over five stream aggregations (source MAC–IP pair, source IP, channel, channel jitter, and socket), including two-dimensional statistics that relate each channel or socket to its reverse direction. It keeps a sliding window of the last ten feature vectors per device and scores each full window with a sequence autoencoder: a two-layer LSTM [23] encoder compresses the 10×115 window into a 64-dimensional state, which is repeated across ten steps and decoded by a second two-layer LSTM and a linear layer (153,651 parameters in total). The anomaly score is the mean squared reconstruction error over the whole window, computed with ONNX Runtime [24] on min–max-scaled features, and an event is raised when it exceeds a single threshold shared by all devices.

C. LLM classification

The classifier service enriches each anomaly event with the device's vendor and known CVEs from the registry and with the reputation of the remote address from AlienVault OTX [25] and AbuseIPDB [26]. It then asks Gemma 4 E2B [27], served locally by Ollama [28] and quantised to 4 bits, for a JSON object with an ATT&CK tactic and technique identifier, a confidence, a severity from 1 to 10, a one-sentence summary and a recommended action. The model runs with Ollama's default sampling (temperature 1.0), so its output for a given event is not deterministic. Events whose response does not parse are dropped.

D. Response and enforcement

The response service is a two-node LangGraph [29] state machine, not a second language model: a policy node maps the LLM's severity to actions through a declarative policy file, and a gate node holds the action for human confirmation when the policy requires it. In the evaluated configuration, severities 6–10 cause the event's indicator address—the destination of the anomalous flow—to be blocked without confirmation, and lower severities are logged. The policy also names a device-quarantine action for severities 9–10; the responder currently enforces it as the same address block, and a MAC-level quarantine module exists but is not invoked by the responder. Keeping the enforcement decision deterministic and inspectable, rather than generated, is deliberate: the model proposes a severity, and a policy the owner can read decides what happens.

Blocks are applied as nftables rules in a dedicated table, each tracked by its rule handle so that it can be removed individually. A guard refuses to block the appliance's own addresses, the default gateway, loopback, broadcast and unspecified addresses, and address ranges, so that a misclassification cannot cut the home off its own router. An XDP fast path is implemented but was not enabled for this evaluation.

E. Deception

Cowrie [5] (SSH and Telnet) and OpenCanary [6] run in containers. A honeypot service follows Cowrie's JSON log, records credentials and commands from each session as a threat event, and sends the attacker's address to the same guarded enforcement path.

F. What leaves the appliance

No packet payload, capture or feature vector is sent off the device, and the LLM runs locally. Three lookups do leave it: the remote address of each anomalous flow goes to OTX and AbuseIPDB, and device vendor names go to the NVD. All three are optional, and a privacy-sensitive deployment can disable them at some cost to context.

G. Threat model

In scope are network-observable attacks involving devices on the monitored segment: compromised devices scanning or flooding (as in N-BaIoT), and inbound credential attacks against exposed services (as seen by the honeypots). We assume the attacker does not control the appliance and cannot tamper with its view of the traffic. Out of scope are physical access, inspection of encrypted payloads, supply-chain compromise, attacks on SENTINEL itself, and adaptive attackers who shape traffic to evade the detector.

IV. Evaluation Methodology

A. Dataset and the deployed model's provenance

We evaluate detection on N-BaIoT [9]: benign traffic from nine commercial devices (doorbells, a thermostat, a baby monitor, security cameras and a webcam) and traffic from the same devices after infection by Mirai and BASHLITE, each executing five attacks. Features are the 115 Kitsune statistics described in §III. The attack classes are defined by the dataset's authors; for example, BASHLITE *COMBO* is "sending spam data and opening a connection to a specified IP address and port" and Mirai *UDPplain* is "UDP flooding with fewer options, optimized for higher PPS" [9].

The model shipped with SENTINEL had been trained in a notebook that was not kept, so its training data was undocumented. We recovered it from the saved min–max scaler: its

115 per-feature minima and maxima match the benign traffic of exactly six of the nine devices with zero deviation, and the number of rows it was fitted on (322,064) equals their combined benign row count. We call these six devices *in-distribution* and the remaining three (*Ennio* doorbell, *Philips* baby monitor, *SimpleHome* XCS7-1003 camera) *held-out*. No model evaluated here saw any traffic from the held-out devices, which therefore measure how the detector behaves on a device it has never observed—the ordinary case for a home appliance.

B. Splits and leakage

Adjacent rows of Kitsune features share damped statistics, so a random train/test split places near-duplicate rows on both sides of the boundary. Following Meidan et al., we split each in-distribution device's benign traffic *chronologically* and keep the final third for testing [9]. We train on the first 90% of the remainder and use the last 10% for early stopping and threshold selection. Scalers are fitted on the training portion only. Attack traffic is never used for training or threshold selection. Windows are ten consecutive rows from a single capture file, never spanning a file or device boundary; test windows do not overlap.

The deployed model does not satisfy these conditions—its scaler saw all benign rows of the in-distribution devices—so we report its in-distribution false-positive rate only as optimistic and rely on its held-out-device and attack results, which are clean.

C. Models compared

- *Deployed*: the shipped ONNX model with its shipped threshold.
- *LSTM-AE*: the same architecture retrained under the protocol above with five seeds. Its parameter count reproduces the deployed model's exactly.
- *Dense AE*: the per-row deep autoencoder architecture of N-BaIoT's authors [9] (hidden widths 86–57–37–28, mirrored), scoring a window by its rows' mean error. It sees the same windows as the LSTM-AE but models no temporal order.
- *Isolation Forest* [30] on the flattened window, as a non-neural baseline.

Every retrained model uses the same threshold rule: the 99th percentile of its own validation-benign scores, a nominal 1% false-positive budget fixed without reference to attack traffic.

D. LLM classification (pre-registered)

To avoid grading our own test cases after seeing the model's answers, we committed the experimental design to the project repository before running the model on any sample. The ground truth maps each N-BaIoT attack class, by its authors' definition, to a set of acceptable ATT&CK techniques: scans to Network Service Discovery, Active Scanning or Remote System Discovery, and all floods to Network or Endpoint Denial of Service. Equivalent techniques from the ATT&CK for ICS matrix are accepted, because SENTINEL's prompt asks for that matrix. We drew 20 windows per attack class (200 in total) with a fixed seed, from both device groups, and queried the model under two prompts: *as-built*, produced by SENTINEL's own prompt builder, and *evidence*, identical but for five plain-language traffic statistics computed from the window (recent packet count, mean packet size, approximate numbers of destination hosts and of sockets, and channel jitter). Fields that could leak the label—the protocol, notably—were withheld from both, and reputation lookups were disabled. We report technique and tactic accuracy with Wilson 95% intervals [31], the rate of valid JSON, and latency. Two further analyses were declared after the initial design but before any output was seen: as a secondary analysis, results restricted to the windows the detector would actually forward to the classifier; and, labelled exploratory, the rate at which the pipeline would have blocked, i.e. the fraction of attack windows assigned severity ≥ 6.

E. Latency and host

We time each stage of the path in process with the real components: feature extraction per packet on real traffic (the Mirai capture distributed with Kitsune [32]), anomaly inference per window, a Redis bus hop, the response policy, and the threat-intelligence lookups. LLM latency is taken from the 90 classification calls that ran on this host. All timing measurements ran on one host: a four-core aarch64 virtual machine with 7.7 GB of memory. This is the same instruction set as, and a similar core count and memory size to, the Raspberry Pi 5 that SENTINEL targets, but it is not that device: it does not reproduce the Pi's clock speed, memory bandwidth or thermal throttling, so absolute timings on the Pi will differ. Neural models were trained on a Colab T4 GPU.

Deviation from the pre-registration. The first 90 of the 400 LLM calls ran on this host as planned; the run was then stopped at a checkpoint because it monopolised the workstation, and the remaining 310 ran on a cloud GPU with the same model (verified by digest), prompts and sampling. We recorded this, and its consequences, in the repository before any cloud output existed: latency is reported from the local calls only, and accuracy pools both hosts, with a per-host check.

V. RESULTS

A. Detection, per attack class

Table I and Fig. 2 report detection per attack class. Both the deployed model and its leakage-free retraining catch eight of the ten classes almost perfectly—every Mirai attack, and BASHLITE's scan, junk and combo—yet miss nearly all of BASHLITE's TCP and UDP floods: the deployed model detects 0.71% and 0.36% of them, and the retrained model $0.31\% \pm 0.09\%$ and $0.15\% \pm 0.05\%$ over five seeds. Because these two classes make up 27.8% of all 650,631 attack windows, the overall detection rate is only 72.4% for the deployed model and $72.29\% \pm 0.02\%$ for the retrained one, a figure that no aggregate accuracy on the remaining classes would hint at.

The miss is not a matter of a badly placed threshold. The retrained model separates the other eight classes from benign

TABLE I
DETECTION RATE (%) PER ATTACK CLASS, ALL DEVICES POOLED. LSTM-AE AND DENSE AE ARE MEANS OVER FIVE SEEDS, ISOLATION FOREST OVER FOUR (THE FIFTH RUN WAS LOST WHEN THE CLOUD VM RUNNING IT WAS RECLAIMED).

Botnet	Attack	Deployed	LSTM-AE	Dense AE	I. Forest
Mirai	scan	99.9	99.9	99.9	0.0
Mirai	ack	100.0	100.0	100.0	41.7
Mirai	syn	100.0	100.0	100.0	0.0
Mirai	udp	100.0	100.0	100.0	36.5
Mirai	udpplain	100.0	100.0	100.0	54.6
BASHLITE	scan	100.0	100.0	100.0	0.0
BASHLITE	junk	100.0	100.0	100.0	0.0
BASHLITE	combo	100.0	100.0	100.0	0.0
BASHLITE	tcp	0.7	0.3	0.0	0.0
BASHLITE	udp	0.4	0.2	0.0	0.0

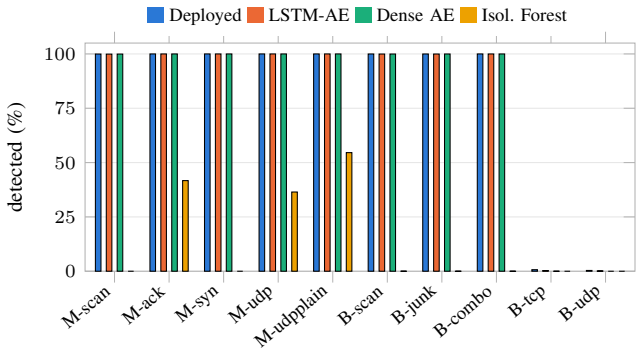


Fig. 2. Detection rate per attack class (M: Mirai, B: BASHLITE). Two BASHLITE flood classes are almost entirely missed; Table I gives the values.

traffic with an AUC of 1.000 ± 0.000, but the two BASHLITE floods with an AUC of only 0.747 ± 0.121. Lowering the threshold far enough to catch half of their windows would raise the false-positive rate to $7.1\% \pm 1.9\%$ on the training devices and $25.5\% \pm 12.2\%$ across all devices. For the deployed model the diagnosis is especially stark: every window of these floods receives almost the same reconstruction error, just below its threshold and inside the range of ordinary benign variation. Setting a separate threshold per device, which we also tried, recovered them on one device only.

The two baselines locate the cause. The dense autoencoder of N-BaIoT's authors, which scores rows independently and models no order, misses the two floods entirely ($0.00\% \pm 0.00\%$ and $0.00\% \pm 0.00\%$), and so does the Isolation Forest; the blind spot belongs to reconstruction- and isolation-based detection on these features, not to the LSTM. Elsewhere the dense model matches the sequence model: the same overall detection ($72.23\% \pm 0.00\%$ against $72.29\% \pm 0.02\%$) and the same false-positive rate on unseen devices ($3.65\% \pm 0.09\%$ against $3.65\% \pm 0.12\%$), with half the false positives on the training devices ($0.11\% \pm 0.01\%$ against $0.23\% \pm 0.02\%$) and a higher AUC on unseen devices (0.958 ± 0.016 against 0.906 ± 0.047). On this data, modelling packet order adds cost without a measurable benefit. The Isolation Forest raises the fewest false alarms on unseen devices ($0.94\% \pm 0.21\%$) only because it detects so little: $15.41\% \pm 7.89\%$ of attack windows at the same nominal budget, although its AUC of 0.977 ± 0.001 shows that its ranking is better than its operating point.

B. Unseen devices

Evaluating on devices absent from training raises the pooled false-positive rate of the retrained model from $0.23\% \pm 0.02\%$ on the training devices to $3.65\% \pm 0.12\%$ on the held-out ones (the deployed model reaches 4.42% on them). The increase is not uniform. The held-out *Ennio* doorbell is flagged as rarely as the training devices, while the *Philips* baby monitor, whose benign traffic is the largest of any device in the dataset, produces most of the false alarms. A single threshold calibrated on a household's existing devices thus says little about the next device added to it, and a single device with unusual traffic can dominate the false-alarm budget of the whole network.

TABLE II
INVARIANTS MEASURED ON N-BAIOT, CHECKED ON REAL TRAFFIC FOR THE ORIGINAL AND THE CORRECTED LIVE EXTRACTOR (SHARE OF PACKETS SATISFYING EACH).

Invariant (from N-BaIoT's rows)	Original	Corrected
$\mathrm{MI}_{\mathrm{dir}} = \mathrm{H}$ (one IP per MAC)	98.70%	99.99%
channel weight $\leq$ host weight	100.00%	100.00%
radius $\geq \mathrm{std}^2$	45.47%	100.00%
magnitude $\geq \lvert\mathrm{mean}\rvert$	100.00%	100.00%
$\lvert\mathrm{pcc}\rvert \leq 1$	100.00%	100.00%
distinct pcc values	2	47,479

C. Live feature parity

Offline results describe the model on N-BaIoT's features; they describe the running system only if the live extractor computes the same features. We checked this with invariants measured on N-BaIoT's own rows, which any faithful extractor must satisfy, on 55,318 real packets from the Mirai capture distributed with Kitsune [32] (Table II). The extractor originally shipped with SENTINEL fed a single stream to both sides of every two-dimensional statistic, so its correlation features took only 2 distinct values and its radius violated the bound on most packets. After replacing it with the reference statistics engine and pinning the stream layout to what the data shows, every invariant holds and the correlation features take 47,479 distinct values within $[-1, 1]$.

D. LLM classification

Table III summarises the pre-registered experiment. With the prompt SENTINEL actually sends, the model named an acceptable ATT&CK technique for 0.0% [0.0, 1.9] of the 200 attack windows and an acceptable tactic for 0.0% [0.0, 1.9], although every response was valid JSON. Adding the traffic summary did not rescue it: 0.5% [0.1, 2.8] for the technique and 4.0% [2.0, 7.7] for the tactic. The pre-registered expectation that evidence would help is therefore not supported at this model size. Restricting to the windows the detector

TABLE III
LLM ATT&CK LABELLING OF 200 ATTACK WINDOWS PER PROMPT (GEMMA 4 E2B). BRACKETS: WILSON 95% INTERVALS.

	As built	With evidence
Technique correct	0.0% [0.0, 1.9]	0.5% [0.1, 2.8]
scans ($n = 40$)	0.0% [0.0, 8.8]	2.5% [0.4, 12.9]
floods ($n = 160$)	0.0% [0.0, 2.3]	0.0% [0.0, 2.3]
Tactic correct	0.0% [0.0, 1.9]	4.0% [2.0, 7.7]
Valid JSON	100.0%	100.0%
Would block (sev. ≥ 6)†	81.0% [75.0, 85.8]	80.5% [74.5, 85.4]

†Exploratory analysis, declared before any output was seen.

would actually forward changes nothing (0.0% [0.0, 2.3] as built).

The failure has a consistent shape. Across all 400 responses, 90.8% name Command and Control as the tactic, typically technique T1071 (Application Layer Protocol), and not one names Impact, the tactic for denial of service, although 320 of them describe floods. Given a device, a remote address and a high anomaly score, the model describes a compromised device talking to a controller—a plausible prior, and wrong for both scans and floods. Its reading of the evidence follows the same prior: a scan's many short connections become "a high volume of connections … potentially indicative of C2 communication." Accuracy did not differ between the calls run locally (0.0% [0.0, 4.1], $n = 90$) and on the cloud GPU (0.3% [0.1, 1.8], $n = 310$).

Enforcement nevertheless happens. 71.0% of responses assign severity 9 or more, and under SENTINEL's policy the pipeline would have blocked 81.0% [75.0, 85.8] of the attack windows as built (exploratory analysis). The block is thus decided by a severity the model assigns almost regardless of what the traffic is, and the ATT&CK label attached to it is nearly always wrong. We did not query the model on benign windows, but nothing in its behaviour suggests it would treat a detector false positive differently.

E. Latency

Table IV gives the median time of each stage. Everything the appliance computes itself is fast: extracting features and scoring a window together take 0.156 ms per packet, enough for roughly 6,400 packets per second on one core, and the bus hop and response policy each cost under a millisecond. Enforcing a decision is also quick: an nftables block takes 16.0 ms at the median (95th percentile 32.1 ms). The path is dominated by two calls. The LLM needs 9.5 s per event (95th percentile 11.7 s), and 44.9 s for the first event after the model service starts. The AlienVault OTX lookup is slower and far less predictable: a median of 21.5 s over 5 lookups, ranging from 7.2 s to 152 s. It is made with a synchronous client inside the classifier's event loop and without a timeout, so while it waits no other event is classified. The classifier reads only the *general* section of the OTX response, yet requests every section; requesting that section alone takes 0.36 s (median of 10), a 60× reduction.

TABLE IV
MEDIAN LATENCY PER STAGE ON A 4-CORE AARCH64 HOST.

Stage	Median
Feature extraction (per packet)	0.073 ms
Anomaly inference (per window)	0.083 ms
Redis bus hop	0.120 ms
Response policy (LangGraph)	0.486 ms
nftables block	16.0 ms
LLM classification (warm)	9.5 s
LLM first call after start	44.9 s
AbuseIPDB lookup	0.73 s
OTX lookup, all sections (as built)	21.5 s
OTX lookup, *general* section only	0.36 s

VI. DISCUSSION

A. Lessons

a) Report detection per class, and per device: An aggregate detection rate on N-BaIoT would have told us that the detector works; it catches almost everything in eight classes. Broken down by class it is blind to two of the ten, and broken down by device its false alarms come mostly from one. Neither fact changes the model; both change whether it is fit to deploy. N-BaIoT's authors trained one model per device and reported that all attacks were detected [9]; a single shared model, the realistic configuration for an appliance, behaves differently, and only a per-class, per-device report shows how. This is the class-imbalance and misleading-metric pitfall described by Arp et al. [8], arising here from structure within the attack traffic rather than from the benign-to-attack ratio.

b) Evaluate the running system, not only the model: A model can be evaluated flawlessly on its dataset while the deployed system feeds it something else. SENTINEL's extractor was written from a description of Kitsune rather than from its code, and the error it contained was invisible in every offline experiment, because offline experiments read N-BaIoT's precomputed features and never touch the extractor. The check that caught it needs no additional data: structural invariants measured on the training set's own rows, asserted on live output. We recommend such parity tests as a standing part of any pipeline that trains on features computed by someone else's tool.

c) Recover what an artefact encodes: The deployed model arrived without its training script. Its scaler recorded, in its per-feature minima, maxima and sample count, enough to identify the exact training devices—which in turn exposed that its in-distribution evaluation would have been contaminated, and supplied a natural held-out-device split. Fitted preprocessing objects are an underused source of provenance.

d) A small local LLM is not yet a classifier: We expected the classifier to fail without evidence and to recover with it. Only the first half held. A 4-bit model with 2.3 billion effective parameters given a structured anomaly event labels almost everything as Command and Control, with or without a description of the traffic. Three consequences follow for designs like ours. First, the ATT&CK label on an alert should

not be trusted, or shown to a user, without validation of the model that produced it. Second, when the label is wrong but the severity is high, a severity-gated policy turns the language model into an expensive way of blocking nearly everything the detector forwards; the anomaly threshold, not the model, is then the real decision point. Third, the privacy argument for running the model locally is sound, but locality constrains model size, and at this size the model does not do the job asked of it. Larger local models, few-shot prompts built from labelled examples, or constraining the output to the techniques consistent with the evidence are the obvious next steps; each should be judged by the same pre-registered test.

e) Latency lives in the integration: Every component we wrote runs in well under a millisecond per event; the detection-to-response path is slow because of an external reputation service called in the wrong way and a language model on a CPU. Neither problem is visible when the detector is benchmarked alone. Requesting only the needed section of the OTX response, running it off the event loop with a timeout, and caching per address would remove most of the reputation cost.

B. Limitations and threats to validity

Dataset. N-BaIoT was captured in 2018 in a laboratory, with nine devices and two botnet families; home networks today carry more devices, more encrypted traffic and other malware. Our results characterise these attacks on these devices. The held-out-device split measures generalisation to three unseen devices of the same era, not to arbitrary new hardware.

Hardware. All timings come from a four-core aarch64 virtual machine, not a Raspberry Pi 5. The host matches the Pi in instruction set, core count and approximate memory, but not in clock speed, memory bandwidth or thermal behaviour, and sustained LLM inference on the Pi will be slower. Physical validation remains future work.

Replay, not deployment. Detection results come from replaying captured features, not from a live network; the parity test narrows but does not close the gap between the two. We have not evaluated SENTINEL against an adaptive attacker.

LLM evaluation. The ground truth maps attack classes to ATT&CK techniques by our reading of the dataset's definitions; other mappings are defensible, which is why we accept several techniques per class and report tactic-level accuracy as well. The model samples at temperature 1.0, so a single run estimates, rather than fixes, its behaviour; 200 windows per prompt give 95% intervals no wider than about ± 7 percentage points.

Deception path. The honeypot-to-firewall path was exercised functionally, by attacking our own honeypot, but is not evaluated quantitatively here; the appliance's own event log mixes those sessions with synthetic events injected to test the response logic, so we do not report counts from it.

Single threshold. Every retrained model uses one global threshold at a nominal 1% false-positive budget; other operating points trade detection for false positives differently, and the threshold-free AUC values we report summarise that trade-off.

C. Ethics

No data about people was collected for this work. Detection experiments use the public N-BaIoT dataset and the Mirai capture distributed with Kitsune. The LLM experiment used reserved documentation addresses (RFC 5737), and reputation lookups in the latency experiment were made only for such addresses. The honeypot path was exercised only by the author, attacking the author's own honeypot from the author's own device.

VII. CONCLUSION

SENTINEL shows that a complete detection-to-response pipeline for a home network, including a language model, runs on a four-core, 8 GB-class host of the kind it targets, and that everything except the language model and one external lookup runs in well under a millisecond per event. Its evaluation shows how much a system can look right while being wrong in specific, consequential ways: a detector that misses two whole attack classes behind a respectable aggregate, false alarms concentrated on one unseen device, a live feature extractor that fed the model inputs it was never trained on, and a local LLM whose ATT&CK labels are almost never correct while its severities still drive blocking.

None of these problems was visible from the headline numbers, and each was found by a check that is cheap to run: report per class and per device, hold out whole devices, test live features against invariants of the training data, pre-register the LLM test, and time every stage of the path rather than the model alone. We offer these checks, and the harness that implements them, as the more lasting contribution. Immediate next steps are to address the BASHLITE flood blind spot with features that capture it, to calibrate thresholds per device class, to make the reputation lookup asynchronous and selective, to replace free-form LLM labelling with a validated or constrained classifier, and to repeat the timing on physical Raspberry Pi 5 hardware.

REFERENCES

[1] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110.

[2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[3] The MITRE Corporation, "MITRE ATT&CK," https://attack.mitre.org, accessed 2026-09-25.

[4] The Netfilter Project, "nftables," https://netfilter.org/projects/nftables/, accessed 2026-09-25.

[5] "Cowrie SSH/Telnet honeypot," https://github.com/cowrie/cowrie, accessed 2026-09-25.

[6] Thinkst Applied Research, "OpenCanary," https://github.com/thinkst/opencanary, accessed 2026-09-25.

[7] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.

[8] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[9] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.

[10] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[11] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[12] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2025.

[13] S. Roy, E. Panaousis, C. Noakes, A. Laszka, S. Panda, and G. Loukas, "SoK: The MITRE ATT&CK framework in research and practice," arXiv preprint arXiv:2304.07411, 2023.

[14] B. Al-Sada, A. Sadighian, and G. Oligeri, "MITRE ATT&CK: State of the art and way forward," arXiv preprint arXiv:2308.14016, 2023.

[15] S. Roy, "AgenticCyber: A GenAI-powered multi-agent system for multimodal threat detection and adaptive response in cybersecurity," arXiv preprint arXiv:2512.06396, 2025.

[16] C. Guan, H. Liu, G. Cao, S. Zhu, and T. La Porta, "HoneyIoT: Adaptive high-interaction honeypot for IoT devices through reinforcement learning," in *Proc. 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec)*, 2023, pp. 49–59.

[17] F. Heiding, M.-A. Omer, A. Wallström, and R. Lagerström, "Securing IoT devices using geographic and continuous login blocking: A honeypot study," in *Proc. 6th International Conference on Information Systems Security and Privacy (ICISSP)*, 2020, pp. 424–431.

[18] H. J. Hadi, M. Adnan, Y. Cao, F. B. Hussain, N. Ahmad, M. A. Alshara, and Y. Javed, "iKern: Advanced intrusion detection and prevention at the kernel level using eBPF," *Technologies*, vol. 12, no. 8, p. 122, 2024.

[19] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, "Real-time intrusion detection and prevention with neural network in kernel using eBPF," in *Proc. 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024, pp. 416–428.

[20] A. Tolay, "eBPF-based real-time DDoS mitigation for IoT edge devices," arXiv preprint arXiv:2508.00851, 2025.

[21] D. Y. Huang, N. Apthorpe, F. Li, G. Acar, and N. Feamster, "IoT inspector: Crowdsourcing labeled network traffic from smart home devices at scale," *Proc. ACM Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 4, no. 2, pp. 1–21, 2020.

[22] E. Lear, R. Droms, and D. Romascanu, "Manufacturer usage description specification," IETF, RFC 8520, 2019.

[23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[24] Microsoft, "ONNX Runtime," https://onnxruntime.ai, accessed 2026-09-25.

[25] "AlienVault Open Threat Exchange (OTX)," https://otx.alienvault.com, accessed 2026-09-25.

[26] "AbuseIPDB," https://www.abuseipdb.com, accessed 2026-09-25.

[27] Google DeepMind, "Gemma 4 model card and technical report," https://ai.google.dev/gemma/docs/core, 2026, accessed 2026-09-25.

[28] "Ollama," https://ollama.com, accessed 2026-09-25.

[29] LangChain, "LangGraph," https://github.com/langchain-ai/langgraph, accessed 2026-09-25.

[30] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[31] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[32] Y. Mirsky, "Kitsune-py: Reference implementation of Kitsune and AfterImage," https://github.com/ymirsky/Kitsune-py, 2018, MIT License. Accessed 2026-09-25.